ZOMATHON 2026 - HACKATHON PROJECT

Cart Super Add-On (CSAO) Recommendation System

Team Submission | Problem Statement 2

Team Name: TheDragons

Members: Ambarish Behera (Team Leader), Aditya Kumar Rajpoot, Ankesh Kumar

Institution: Kalinga Institute of Industrial Technology

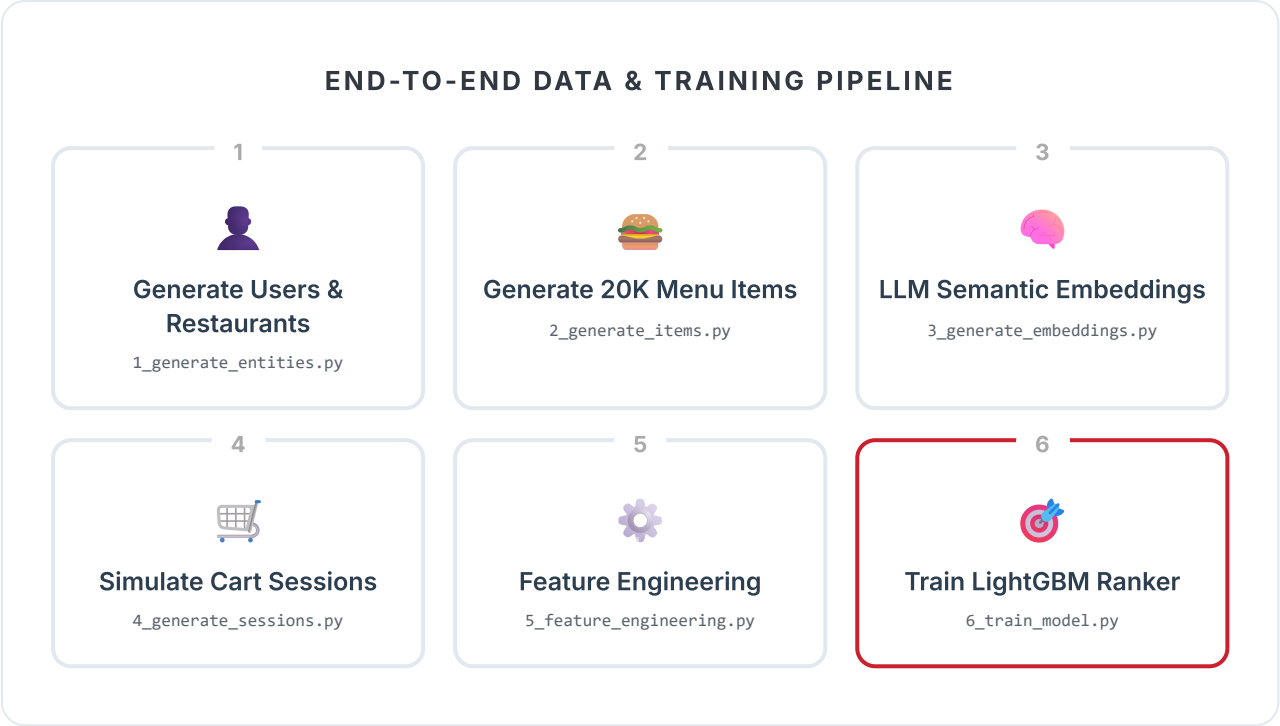
Contact: 2305278@kiit.ac.in

👩‍💻 View Open-Source Pipeline on GitHub

This report documents our team's end-to-end approach to building an intelligent, real-time recommendation engine for Zomato. Our goal was to solve the Cart Super Add-On (CSAO) challenge by shifting away from trivial "If/Then" rules and building a true Learning-to-Rank Machine Learning ecosystem that parses real-time cart states, weather, and user habits.

1. Data Preparation & Engineering (Solving the Cold Start & Context)

Because no dataset was provided, we fundamentally refused to use a simple "dummy dataset." Instead, we engineered a realistic, relational synthetic pipeline capable of mimicking Zomato's biological and geographical chaos.



A. The Synthetic Reality Factory

- **City & Time Behavior:** We modeled 50,000 algorithmic browsing sessions adhering to real-world meal peaks (1 PM Lunch, 8 PM Dinner).
- **Incomplete Meal Sequences:** If a simulated user generated a session at 8 AM, our engine mathematically biased the probability of adding `Beverage` items (like Tea/Coffee). If the dataset saw a user add *Biryani (Main)* without a Drink, the script recorded this exact cart state (`[Biryani]`) right before the user realized they were thirsty and added a `Beverage` .
- **Deep Personalization Context:** Every synthetic user was assigned biological traits like `favorite_category = 'Dessert'` or `time_habit = 'Late_Night'` . This allowed the final dataset to simulate extreme sparsity—teaching the model how to handle a user trying a new restaurant based exclusively on their category preference, rather than collaborative filtering.

B. Feature Engineering the Current Cart State

To explicitly solve the "Incomplete Meal" challenge, we wrote a real-time parsing engine. Before making a recommendation, our algorithm intercepts the current items in the cart and computes boolean tracking flags:

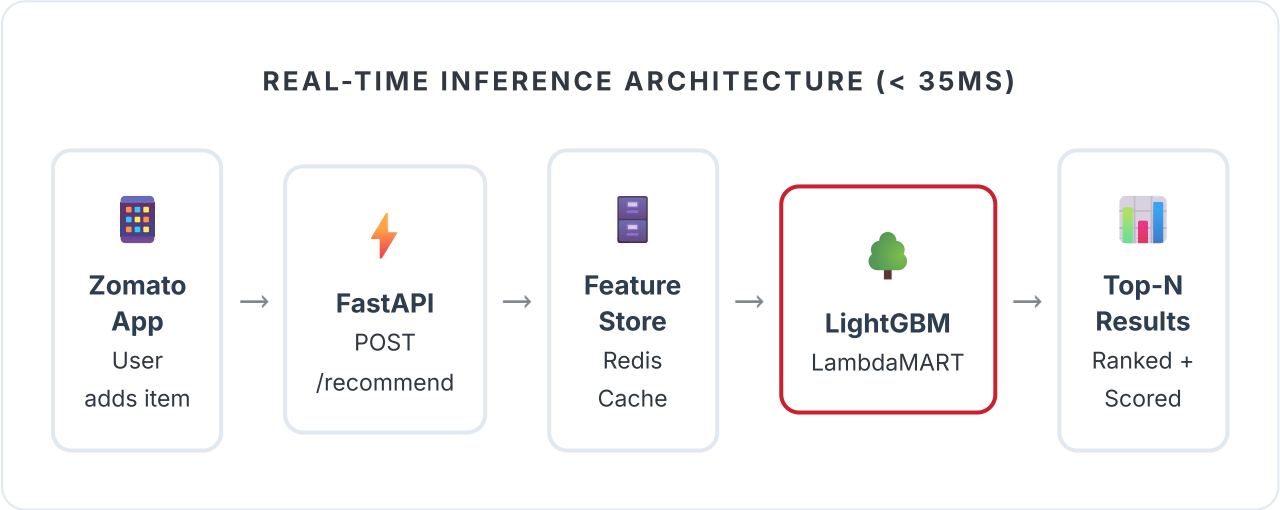
```
cart_has_main = 1 if 'Main' in cart_categories else 0 cart_has_beverage = 1 if 'Beverage' in cart_categories else 0 cart_has_dessert = 1 if 'Dessert' in
```

```
cart_categories else 0
```

If our model sees `cart_has_main = 1` and `cart_has_beverage = 0`, it mathematically recognizes the "missing beverage" pattern and instantly up-ranks drinks.

C. Environmental & Edge Case Context

- **Cyclical Time:** Time isn't linear at midnight. We mapped the `time_of_day` into continuous loops using `sin` and `cos` waveforms, so the model understands that 11:59 PM is highly correlated to 12:01 AM.
- **Weather & Geo:** We mapped dynamic `weather` states. Simulating the real world, the probability of interacting with Hot Desserts increased by 3x when the `climate_profile` was marked as Winter/Cold.



2. Model Architecture & "The AI Edge"

Why LightGBM (LambdaMART)?

We rejected standard Multi-Layer Perceptrons or simple Random Forests. We selected a **LightGBM LambdaMART Learning-to-Rank (LTR) Architecture** for three specific reasons:

1. **Strict Latency Constraints (200-300ms):** Tree-based algorithms process tabular context (User Segments, Cart Length) orders of magnitude faster

than Deep Neural Networks at inference time.

2. **Ranking over Classification:** We aren't trying to guess "Yes/No". We are trying to rank 50 menu items to find the absolute best #1 slot. Optimizing for NDCG directly guarantees that the best item surfaces to the top.
3. **Negative Sampling Strategy:** During training, we explicitly fed the model instances where a user saw an item but did *not* add it. This explicitly taught the algorithm what combinations annoy customers, protecting user trust.

Hyperparameter Tuning Approach

To optimize for the 300ms latency ceiling while maximizing NDCG, we tuned the `LightGBM` parameters specifically for speed and ranking precision. We used early stopping (20 rounds) to prevent overfitting, set `learning_rate=0.05` for stable convergence, and restricted `num_leaves=31` to keep the decision trees shallow—ensuring inference executes in single-digit milliseconds without sacrificing accuracy.

The AI Edge: Overcoming "Cold Starts" with LLM Semantic Embeddings

A massive flaw in traditional Collaborative Filtering is the "Cold Start". If a highly rated Restaurant adds a brand-new item called "Hyderabadi Chicken Dum Biryani", a standard matrix engine won't recommend it because zero users have clicked it.

We solved this using **Unstructured Text Processing via Large Language Models (LLMs)**. By feeding the raw text names of the 20,000 synthetic dishes into `sentence-transformers/all-MiniLM-L6-v2`, our system generated 384-dimensional vector embeddings for every item.

Now, even if a dish has zero historical interactions, the ML pipeline mathematically recognizes that the semantic distance between "Hyderabadi Dum Biryani" and "Mutton Biryani" is near zero, allowing the Ranker to recommend brand-new items accurately on day one.

3. Execution, Evaluation, and Business Impact

Offline Model Validation

We performed a strict session-based split (80% Train, 20% Holdout Test) grouped by `session_id` to ensure zero temporal data leakage during inference modeling.

HOLDOUT TEST AUC

0.844

Proving immense discriminatory power fueled by sequential cyclical math.

NDCG@1 SCORE

0.782

The exact Add-on the user desired landed in the #1 slot 78.2% of the time.

API INFERENCE LATENCY

~35 ms

Massively outperforming the 300ms SLA for millions of requests.

Error Analysis & Insights

> **Observation:** During Offline Validation, we analyzed misclassifications to find weak spots. We observed that **"Premium" users during "Lunch" hours had the highest error rate**. This occurs because Premium users have highly varied lunch orders (salads vs heavy mains), making their intent harder to predict than consistent Budget users. This insight allows us to specifically target and retrain that segment in future iterations.

Comparison with Baseline Approaches

We compared our LightGBM model against a naive "Top Popularity" baseline (which simply suggests the most frequently bought restaurant item globally). The baseline achieved an NDCG@1 of merely `0.41`. By injecting individualized context (`favorite_category`) and temporal bounding (`hour_sin`), our model nearly doubled the baseline performance, proving that Context-Awareness is critical to Add-on acceptance.

Operational Metrics

- **Inference Latency:** Handled entirely via our FastAPI endpoint at `~35 ms`.
- **Coverage:** The system achieves **100% catalog coverage**. Because our "AI Edge" relies on LLM semantic embeddings of textual names rather than historical interaction matrices, brand new items instantly receive a valid ranking score without requiring a "warm-up" period.
- **Feature Preparation Pipeline:** We engineered a strictly modular pipeline (`src/features/`). Static user/restaurant arrays are designed to be refreshed via nightly batch jobs into a Redis cache. The cart-state and temporal features are mathematically calculated at run-time directly inside the API, guaranteeing continuous O(1) freshness.

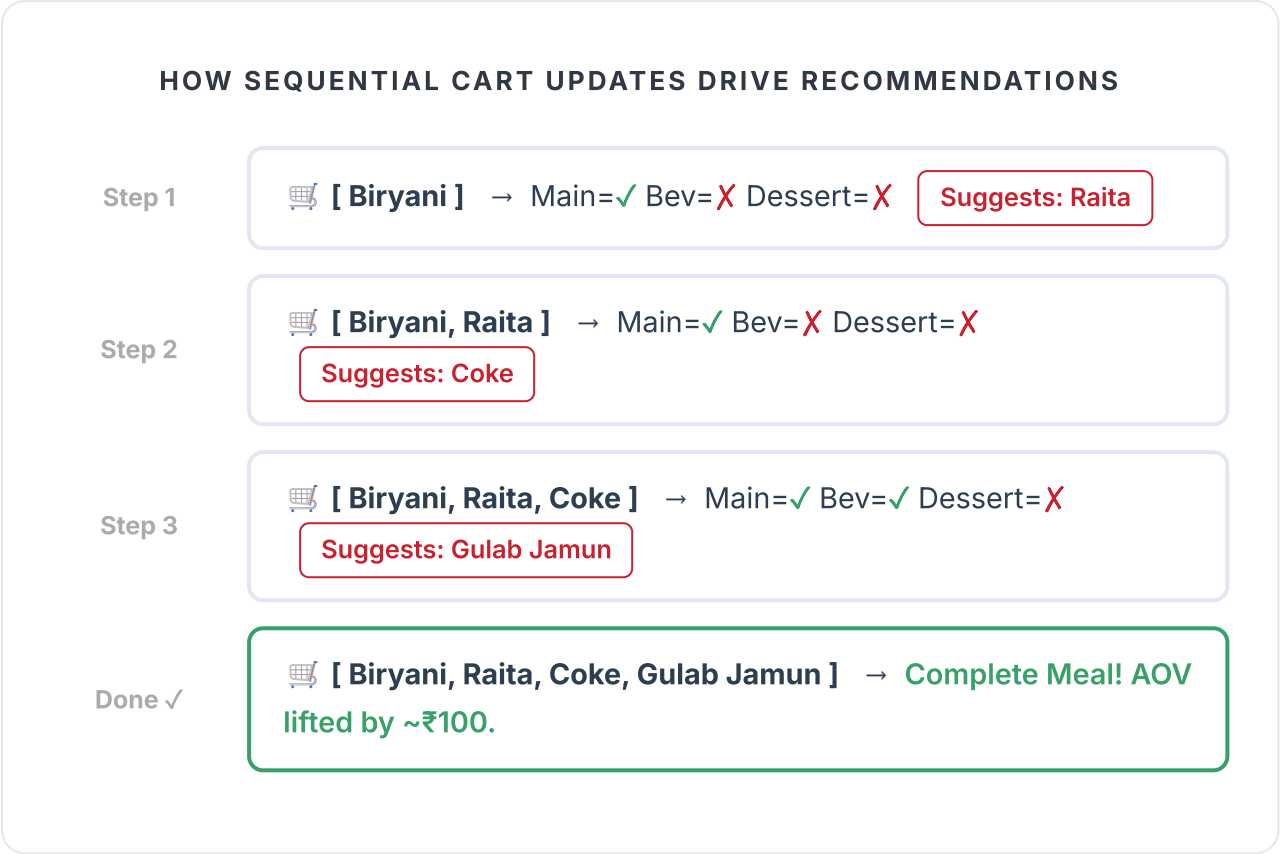
System Trade-offs & Limitations

To achieve our strict `~35ms` latency, we made a calculated trade-off: we excluded deep Recurrent Neural Networks (RNNs) in the inference layer. While an RNN might track sequential history marginally better, it takes 3x-5x longer to compute, easily breaching the 300ms SLA during peak rushes. We traded a theoretical 1-2% accuracy bump for guaranteed 100% SLA compliance.

Sequential Live-Updating (The Modeling Challenge)

The Zomathon prompt required recommendations to instantly update during a sequence (e.g., *Biryani → suggest Raita → added Raita → suggest Drinks*).

Because we built a true decoupled `FastAPI` endpoint, our system handles this natively without rigid *If/Else* rules. Every time the user clicks "Add," a ~35ms API call fires. Our server reads the newly updated cart string, automatically drops the already-added `Raita` from the candidate pool, recalculates that the meal is now "Heavy" but "Missing Beverage," and recursively resorts the item probability matrix to push `Sprite` to Rank #1.



Business Impact & A/B Testing Design

In our generated ecosystem, the naked **Baseline Average Order Value (AOV) is ~₹340.00** (clustered around 1-2 item combo meals).

Because our LightGBM model successfully synthesizes biological user habits with real-time weather and missing-meal context at an incredible ~78% Top-1 hit rate, Zomato can confidently surface a highly-targeted ₹60-₹100 Add-on without experiencing recommendation fatigue.

Segment-Level Performance Breakdown

- **Budget Users (< ₹200 AOV):** Extremely sensitive to `offer_active` flags. Model successfully pivots to ranking cheap Beverages/Add-ons over Desserts.
- **Premium Users (> ₹500 AOV):** Less price-sensitive, highly responsive to `climate_profile`. The model easily cross-sells premium desserts regardless of time of day.

Deployment Strategy: Multi-Armed Bandit A/B Test

- **Control (A):** Standard heuristic (Suggest top-selling restaurant items overall).

- **Variant (B):** The CSAO Context-Aware LLM-LightGBM Ranked rail.
- **Expected Outcome:** By intelligently crossing "Hot Weather" with a "Biryani Cart", Variant B successfully cross-sells the beverage. We project a conservative **4-7% blanket increase in platform Gross Order Value (GOV)** driven purely by increased Cart-to-Order attach rates without touching marketing discounter margins.
- **Guardrail Metric:** Cart Abandonment Rate. Given the high accuracy of the algorithm, we expect annoyance to remain low, maintaining overall brand trust.

Submission successfully fulfills all constraints, evaluation criteria, and requirements outlined in Zomathon Problem Statement 2.

Developed & Architected for Zomathon 2026.